1.Aim of the practical: Create and consume Restful web services for accessing employee data application securely.

2.Objective: The primary objective of this project is to design, implement, and consume RESTful web services for managing employee data in a secure, scalable, and efficient manner.

3.Algorithm/Flowchart:

Node.js:

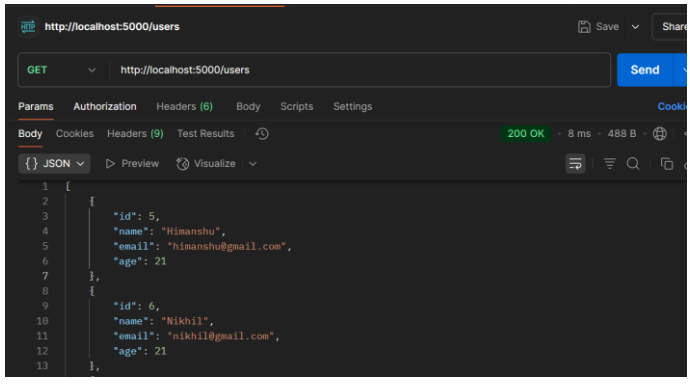
```
const express = require("express");
const bodyParser = require("body-parser");
const cors = require("cors");
const mysql = require("mysql2");
require("dotenv").config();

const app = express();
const PORT = process.env.PORT || 5000;
// app.listen(PORT, () => console.log(`Server running on port ${PORT}`));

// Middleware
// app.use(cors());
app.use(cors({ origin: "http://localhost:3000" })); // React runs on port 3000
app.use(bodyParser.json());

// MySQL Database Connection
```

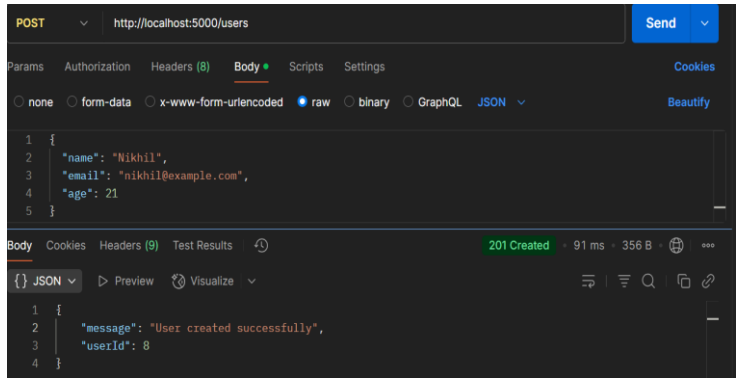
```
const db = mysql.createConnection({
    host: process.env.DB_HOST,
    user: process.env.DB_USER,
    password: process.env.DB_PASSWORD,
    database: process.env.DB_NAME,
});

// Connect to MySQL
db.connect((err) => {
    if (err) {
        console.error("❌ Database connection failed:", err.message);
        process.exit(1); // Stop server if DB connection fails
    }
    console.log("✅ Connected to MySQL database.");
});
// CREATE User (INSERT)
app.post("/users", (req, res) => {
    const { name, email, age } = req.body;
    if (!name || !email || !age) {
        return res.status(400).json({ error: "All fields are required." });
    }
    const sql = "INSERT INTO users (name, email, age) VALUES (?, ?, ?)";
    db.query(sql, [name, email, age], (err, result) => {
        if (err) return res.status(500).json({ error: err.message });
        res.status(201).json({ message: "User created successfully", userId: result.insertId });
    });
});

// READ All Users
app.get("/users", (req, res) => {
    db.query("SELECT * FROM users", (err, results) => {
        if (err) return res.status(500).json({ error: err.message });
        res.json(results);
    });
```

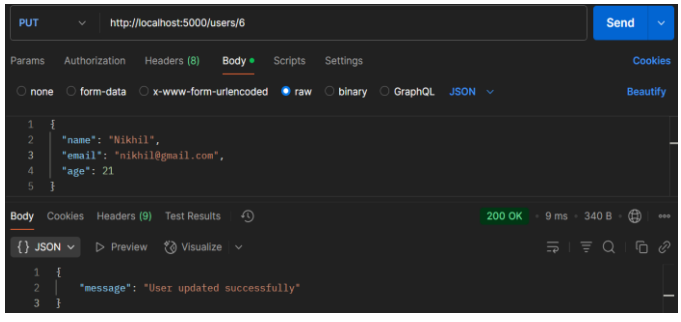
```
});

// READ Single User
app.get("/users/:id", (req, res) => {
    const { id } = req.params;
    db.query("SELECT * FROM users WHERE id = ?", [id], (err, result) => {
        if (err) return res.status(500).json({ error: err.message });
        if (result.length === 0) return res.status(404).json({ message: "User not found" });
        res.json(result[0]);
    });
});

// UPDATE User
app.put("/users/:id", (req, res) => {
    const { id } = req.params;
    const { name, email, age } = req.body;
    if (!name || !email || !age) {
        return res.status(400).json({ error: "All fields are required." });
    }
    const sql = "UPDATE users SET name = ?, email = ?, age = ? WHERE id = ?";
    db.query(sql, [name, email, age, id], (err, result) => {
        if (err) return res.status(500).json({ error: err.message });
        if (result.affectedRows === 0) return res.status(404).json({ message: "User not found" });
        res.json({ message: "User updated successfully" });
    });
});

// DELETE User
app.delete("/users/:id", (req, res) => {
    const { id } = req.params;
    db.query("DELETE FROM users WHERE id = ?", [id], (err, result) => {
        if (err) return res.status(500).json({ error: err.message });
        if (result.affectedRows === 0) return res.status(404).json({ message: "User not found" });
        res.json({ message: "User deleted successfully" });
    });
```

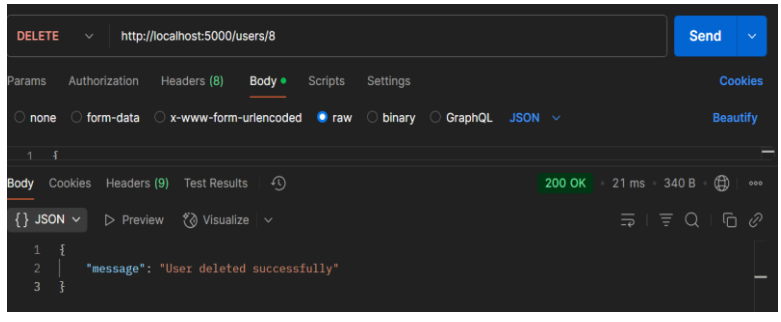
```
});

// Start Server
app.listen(PORT, () => {
  console.log(`🚀 Server running on http://localhost:${PORT}`);
});
```

4.Result/Output:









Learning outcomes (What I have learnt):

- Understand and implement RESTful APIs using technologies such as Node.js, Express, Java, or Spring Boot.
- Perform CRUD operations on employee data via standardized HTTP methods (GET, POST, PUT, DELETE).
- Use tools like Postman or cURL to test API endpoints.